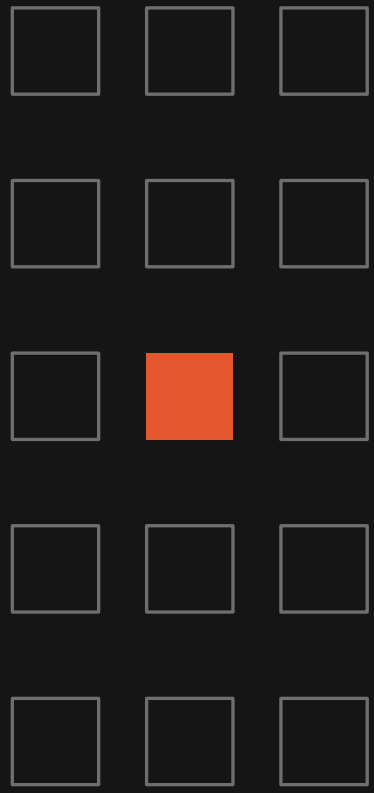
Letting an LLM steer equality saturation, without trusting it

Group research proposal: the gap, our implementation, and the work to be done



one waypoint chosen by the LLM;
every step around it proved

THE PROBLEM

Equality saturation removes phase ordering, but explodes in two places

An e-graph keeps every equivalent rewrite of a program at once. Nothing is deleted, so no rewrite order is ever wrong; extraction then picks the cheapest program.

01

Exploration blows up

Associativity and commutativity alone produce every bracketing of a sum. Node limits are hit before a useful rewrite 10+ steps away is reached.

02

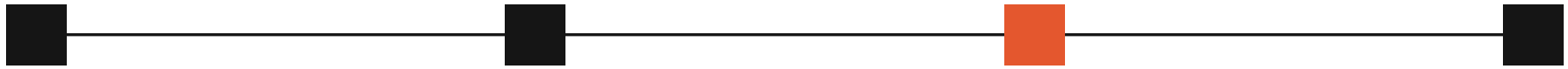
Extraction is NP-hard

When shared subterms are costed once, picking the optimum is NP-hard. Exact ILP solvers time out on large e-graphs.

Until recently both were tamed by hand: rule schedules, sketches, and human-written guides (Koehler et al., Guided Equality Saturation, POPL 2024).

THE IDEA WE BUILD ON

LGuess: the LLM chooses waypoints, the e-graph proves every step



Saturate

Fresh e-graph from the current program

Sample

Candidate terms from a bigram model over operators

Ask the LLM

Which of two terms makes more progress?

Checkpoint

Best term seeds the next phase

Repeat until the checkpoint stops changing. egg's explanation replays the whole chain, rule by rule.

255

of 320 solved by LGuess

204

by plain equality saturation (egg)

3

when the LLM writes the chain itself

Polynomial factorization, GPT-4o. Hardest cells (degree and variables ≥ 4): 30 vs 2. Peng, Ji, Xiong, arXiv 2511.00403 / EGRAPHS 2025.

WHERE THE GAP IS

The field has one empty corner: steering real code with a formal guarantee

LLM steers search

LLM writes code

Steers search · tested

ACCLAIM orchestrator
Magellan (LLVM heuristics)
AwareCompiler

Steers search · formally sound

LGuess (polynomials only)
EggMind (vectorization, XLA)
ASPEN (RTL datapaths)

GAP
real IR + proof gate

Writes code · tested

ACCLAIM level agents, SuperCoder
LLM Compiler, KernelBench
AI CUDA Engineer (gamed its tests)

Writes code or rules · formally checked

LLM-VeriOpt (Alive2 reward)
LPG (peephole rules)
LLM-Vectorizer

Checked by tests

Checked by proof

THREE GAPS

What the current papers leave open

G1

Missing controls

No LLM-guided EqSat paper compares against human guides, a strong scheduler (CERES), or RL/MCTS guidance.

WHAT WE DO

Run the missing control: same phases, cheapest term as checkpoint, no LLM.

G2

Unmeasured cost of guidance

Staged search can prune away the optimum. No paper reports the optimality gap or LLM calls per solved task.

WHAT WE DO

Compare against exact ILP extraction on instances small enough to saturate.

G3

Rules and checkpoints never meet

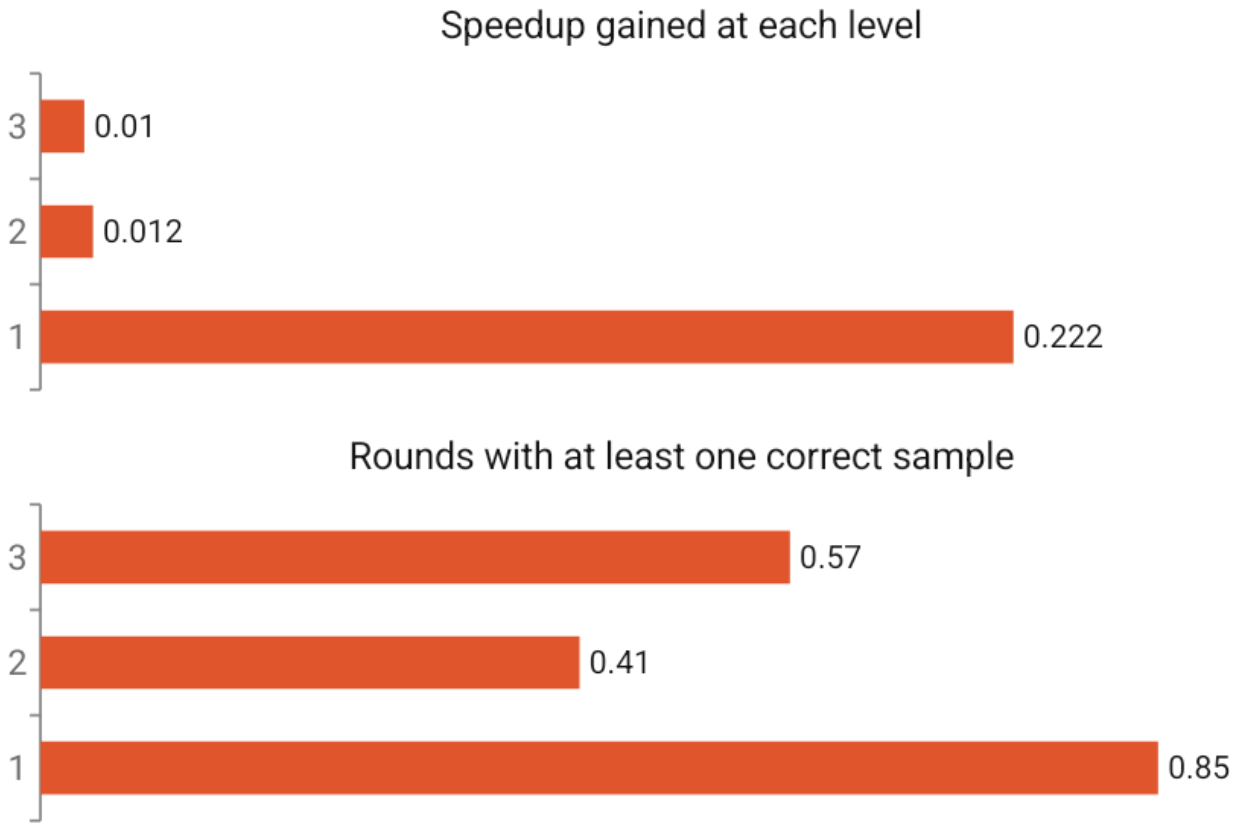
LLMs propose rules (LPG, LLM-VeriOpt) or checkpoints (LGuess), never both.

WHAT WE DO

When a checkpoint is unreachable, ask for a bridging rule; admit it only if Z3 proves it.

SUPPORTING EVIDENCE

The agentic side agrees: LLM edits at IR level barely help



WHY IT MATTERS

The IR wins ACCLAIM did find were compiler-reachable: ×7 became shift-by-3 and subtract.

An e-graph with the right cost model finds those soundly. IR is where e-graphs are strong and LLMs are weak.

ACCLAIM (Mikek et al., AWS AI, arXiv 2604.04238): 1.25× geometric-mean speedup over clang -O3 on CodeNet C; correctness checked by tests only.

OUR QUESTIONS

Four research questions, answered by one harness

RQ1	Does LLM checkpointing beat phased-greedy, human guides, backoff and pulsing schedulers, and MCTS at equal budget?	G1
RQ2	On instances small enough to saturate, how far is the guided result from the true optimum?	G2
RQ3	How many LLM calls per solved task, and can a distilled scorer replace most of them?	G4
RQ4	Does it transfer to straight-line bitvector code, and do Z3-proved bridging rules raise the solve rate?	G5 + G3

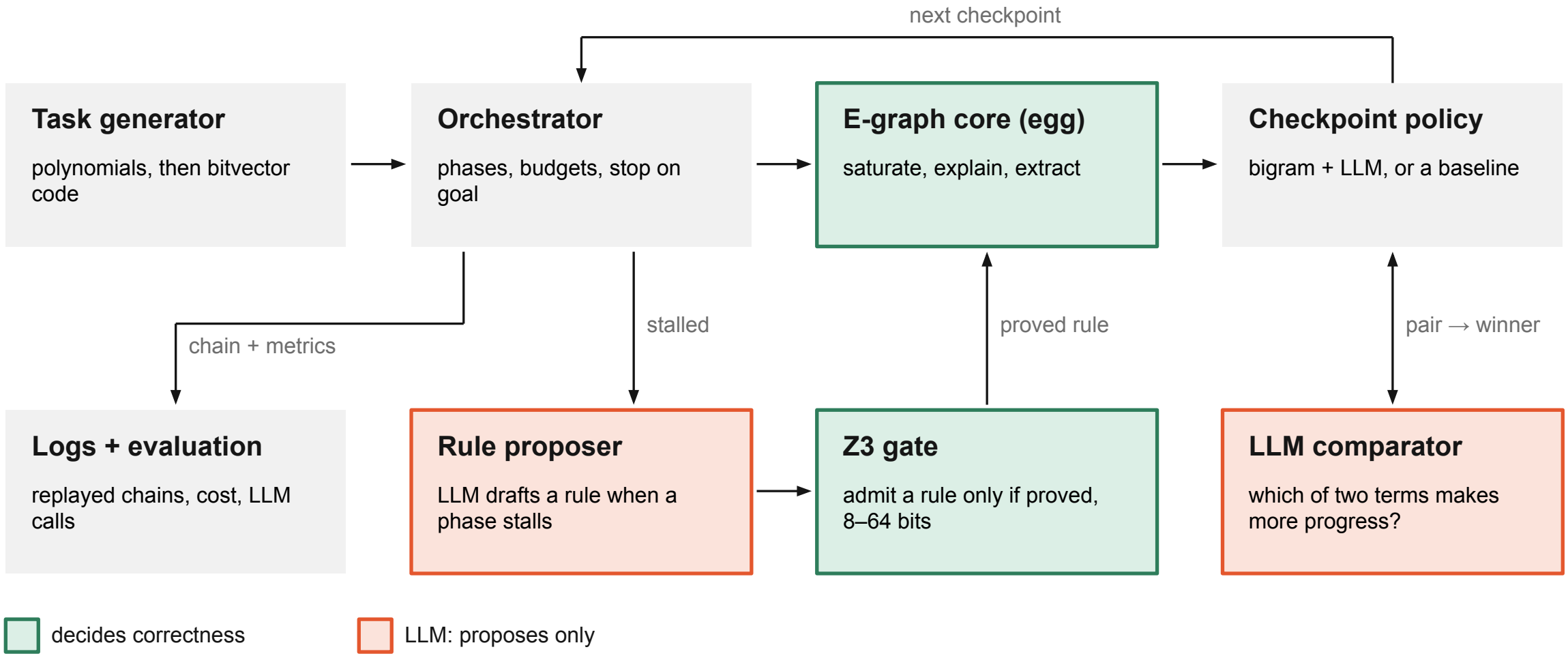
THE CONTROL THAT DECIDES RQ1

Phased EqSat with the cheapest term as checkpoint: same phases, same budget, no LLM. LGuess never ran it. If it matches, the gain came from phasing; either outcome is publishable.

Status: literature review and design are done; implementation has not started.

OUR IMPLEMENTATION

Architecture: the LLM proposes, only proofs decide



OUR IMPLEMENTATION

Our stack, and eight baselines in one harness

STACK

Rust + egg	e-graph core with explanations
pyo3 + maturin	Python bindings to the core
Python	orchestration, models, evaluation
LLMs	one frontier API model, one open 7B model
Z3	rule soundness at 8–64 bits
ILP extractor	true optimum for RQ2
JSONL logs	every table regenerates from runs

Baseline	Question it answers
Direct EqSat	The original comparison
Phased-greedy (no LLM)	Is the gain phasing or judgment?
Phased-random	Floor for any checkpoint policy
Backoff scheduler	Strong default schedule
Pulsing (Caviar-style)	Automatic staging
Human guides (written by us)	Does an LLM beat a person?
MCTS over rule choice	Pre-LLM learned guidance
LLM writes full chain	LLM-only floor (LGuess: 3/320)

WORK TO BE DONE

Five work packages, with two decision points

A	**Build the harness**	egg core with polynomial language and rules · explanations and JSON serializer · Python bindings · LGuess-style task generator (320 tasks)
B	**Reproduce LGuess**	bigram model with sampling and most-probable extraction · LLM comparator with call logging · phased loop · compare against 255/320
◆	**Decision: is our reproduction close to 255/320? If not, we report the gap and still run the baseline study.**	
C	**Baseline study**	all eight baselines at equal budget · optimality gap against exact ILP · LLM calls per solved task · distilled scorer to cut calls
◆	**Decision: the extended abstract for the core study is drafted before any extension work.**	
D	**Extend to bitvector code**	straight-line 32-bit language · Z3-verified rule set · port every method · LLM bridging rules admitted only if Z3 proves them
E	**Write up and release**	paper · one-command reproduction script · task suite and run logs

HOW WE EVALUATE

Evaluation protocol and the risks we expect

PROTOCOL

- Equal compute budget per task; LLM latency counts
- Three seeds per method; LLM at temperature 0
- Solve rate, final cost, optimality gap, peak e-graph size, LLM calls and spend
- Paired per-task comparison; McNemar's test on solve rates
- Every solution replays through egg; every rule passed Z3

Risk	Our mitigation
Reproduction misses 255/320	Report the gap; the baseline study still stands
Phased-greedy matches LGuess	That becomes the RQ1 headline
API cost grows	Per-run spend logs; distilled scorer (RQ3)
Bigram model fails on bitvector code	Expected; motivates a better representation
Scope creep into MLIR or real LLVM	Out of scope until the core study is complete

DELIVERABLES

What we will deliver

1 **The first equal-budget comparison of LLM, human, scheduler, and MCTS guidance for equality saturation**

Answers RQ1 and closes G1

2 **Measured optimality gap and LLM cost per solved task**

Answers RQ2 and RQ3; closes G2

3 **Proof-gated LLM bridging rules on bitvector code**

Answers RQ4; closes G3

4 **An open harness, task suite, and run logs**

So every result can be reproduced and extended

Possible follow-on: constant-time-preserving rewriting, connecting this harness to prior LeakSurface work on side-channel hardening.

ABSTRACT

The core idea: LLMs propose, proofs decide

Equality saturation optimizes programs soundly by exploring every equivalent rewrite at once in an e-graph, but its search explodes before long rewrite chains are found. Recent work lets a large language model steer this search by choosing intermediate checkpoints, while the e-graph supplies the provably sound steps between them. These results are promising but rest on a single synthetic domain and weak baselines.

We build an open harness that compares LLM guidance against human guides, strong schedulers, MCTS, and a no-LLM phased control at equal budget, and that measures the optimality gap and LLM cost of guidance. We then extend the approach to straight-line bitvector code, where the LLM may also propose bridging rewrite rules that are admitted only after Z3 proves them sound. Throughout, the LLM only proposes; correctness is always decided by a proof.